

Vera x Daytona onsite tests — plan for NVIDIA

Status: Ready to share

Owners (Daytona): Ivan, Vedran, Goran, Daniel

Audience: NVIDIA Vera / partnership contacts (Tom, Diana, Ananya, Ian, et al.)

Date: 2026-08-14

Goal

We want to know whether **Vera makes Daytona’s sandbox product clearly better for the workloads our customers run** — coding agents, data/analytics jobs, RL-style rollouts, and eval / Terminal-Bench-style trials — such that **ROI and UX improve** (faster completion, higher concurrent density, more predictable latency).

Daytona software still owns routing, snapshotting, portability, isolation/security, and scheduling. What we want from Vera is **evidence that the same Daytona paths run better on Vera than on today’s ARM64 / x86 regions**, so we can justify preferring Vera capacity for agentic tenants.

What we’d test on Vera

1. **Sandbox lifecycle** — create → exec → delete (and warm reuse where we already support it)
 2. **Snapshot boot path** — region-local snapshots / images for our workload packs
 3. **Concurrent multi-tenant density** — many isolated sandboxes at once (customer-shaped ladders; we already validate through ~88, and are extending toward 100–1k where capacity allows)
 4. **In-sandbox customer compute** — the work *inside* the sandbox (we separate this from API / create round-trips)
-

Prioritized tests

Priority	Test	Why it matters
P0	RL / agent-style rollout CPU — heavy in-sandbox episode; compare time <i>inside</i> the sandbox	Isolates silicon quality : “Vera cores finish the same episode faster.”
P0	Sandbox density — light RL + coding-agent + eval / TB-style trials at rising concurrency	Isolates product density / UX : more concurrent customer jobs with usable tail latency. Closest to how eval / Terminal-Bench-on-Daytona load looks.

Priority	Test	Why it matters
P1	Analytics (Parquet + DuckDB)	Memory-bandwidth-heavy tenants; keep if Vera wins, otherwise appendix.
P1	Media (FFmpeg transcode)	Non-Python bandwidth sibling (agent media preprocess); promote if analytics is flat.
P1	Cold create / schedule tax	Separately measure spin-up (API, scheduling) so it isn't confused with chip time.
P2	Harbor Terminal-Bench oracle pack	"TB oracle pack finishes sooner / denser on Vera" — infra time-to-finish at fixed concurrency.

How we compare (apples-to-apples)

Same snapshots, same seeds / checksums, same concurrency ladder on:

- **Control:** today's Daytona / RLP region (e.g. current ARM64 test region and/or x86)
- **Vera:** Vera as a dedicated Daytona / RLP region

We track two stories separately:

- **Chip:** time spent *inside* the sandbox (duration_ms) — not wall time that includes create/API
- **Product density:** throughput and p99 end-to-end latency when many sandboxes run at once

What we want hard numbers on

- In-sandbox work is **faster on Vera**, not just better network to that region
- At high concurrency, **throughput goes up** and **p99 stays usable**
- **Fewer failures / flaky capacity** under load
- Warm reuse (create once, many execs) shows wall time tracking in-sandbox compute

What "win" looks like

A clear, publishable gap on Vera vs control that we can take into GTC / partnership messaging — without over-claiming if chip or density doesn't move.

Decision metrics (3–5):

1. **In-sandbox time p50 / p99** on a heavy RL episode — Vera vs control (**chip**)
2. **Throughput** (trials or episodes / sec) at high concurrency (**density / ROI**)

3. **p99 end-to-end latency** at peak concurrency (**UX / predictability**)
 4. **Failure / incomplete rate** under the density matrix (**reliability**)
 5. **Time to finish a fixed eval / TB-oracle pack** at fixed concurrency (**customer-facing eval UX**)
-

Asks of NVIDIA

- Confirm **Vera region access** (name / target) for Daytona / RLP onsite runs
- Feedback on this priority order or anything you'd like us to emphasize technically

We'll follow with a detailed run matrix and exact commands once the Vera target is locked. Happy to take comments from Tom / Diana / Ananya / Ian on format or focus.